

On the Computational Complexity of Tile Set Synthesis for DNA Self-Assembly

Xiaojun Ma and F. Lombardi

Abstract—DNA self-assembly has been advocated as a bottom-up manufacturing technology to supersede photolithography technology at nanometer scale. However, the issue of designing a DNA tile set for an arbitrary target pattern of finite size (as to ensure periodic repetition in its assembly) has not been fully addressed in the technical literature. This paper considers the synthesis of tile sets for DNA self-assembly and analyzes it as a combinatorial optimization problem to establish its computational complexity. This problem is referred to as PATS (Pattern Assembling Tile-set Synthesis). A proof is provided for the NP-completeness of PATS.

Index Terms—Algorithmic self-assembly NP-completeness, DNA Self-assembly, nanoscale manufacturing.

I. INTRODUCTION

As per the scaling projections [1], the channel length of today's CMOS transistors is in the order of the width of one hundred atoms. Additional reductions in the feature length of CMOS (such as encountered at 32 nm) are facing fundamental limitations. To meet the ever increasing device density required by current and future computing systems, emerging technologies with small feature sizes are being investigated. Along with new devices, novel technologies for fabrication at nanometer scale are needed. As widely reported in the technical literature, self-assembly using *tiles* made of engineered DNA strands is a promising technology for "bottom-up" manufacturing such as for nanoscale structures [2].

For manufacturing, DNA self-assembly relies on a set of tiles to target a specific pattern. To reduce fabrication/laboratory costs, it is desirable for DNA self-assembly to utilize a tile set of small cardinality. It has been reported in [3] that an arbitrary pattern can be generated using a DNA self-assembly process. Therefore, the capability to generate finite-size arbitrary patterns is highly desirable because these types of patterns are likely to be used to establish the geometry of circuits in nano-scale manufacturing. Synthesized tile sets for these patterns are a necessity for a practical application of DNA self-assembly [4]. This objective has met considerable attention. The tile set used in [3] gives every tile in the aggregate a unique tile type, hence the number of tiles in the tile set is equal to the number of pixels in the pattern, thus incurring in a large

complexity. Such a set is referred to as to *trivial tile set* in this paper. A process referred to as *tile synthesis* has been proposed to simplify the trivial tile set and reduce its cardinality [4].

This paper deals with set synthesis by minimizing the numbers of tile or bond types. The optimization problem for tile set synthesis [4] is referred to as Pattern self-Assembly Tile-set Synthesis (PATS). The main objective of this paper is to assess the computational complexity of the PATS problem. A graph model is used to analyze this problem. By relating PATS to the coloring problem, it is proved that PATS is a NP-complete problem.

II. REVIEW OF DNA SELF-ASSEMBLY

The programmability of DNA in the form of tiles can be exploited to facilitate manufacturing of circuits based on single-molecule electronic devices, such as molecular transistors or quantum-dot cellular automata (QCA) cells [5]. The assembly of DNA structures by a controlled pattern has been demonstrated in [3], [6]. The use of DNA to build nanostructures for molecular electronics has also been reported [7]. The basic principle is to utilize the programmability of DNA tiles to self-assemble periodic and aperiodic lattice structures [7]; then, it is possible for other molecular electronic devices to selectively attach to the lattice. An obvious advantage of this technique is that a massive parallel assembly is possible, such that millions of copies of the desired structure can be built simultaneously [7]. This technique has been advocated as a method for "bottom-up" nanofabrication. As commonly assumed in the technical literature, the *abstract Tile Assembly Model (aTAM)* provides the basis for analysis of algorithmic self-assembly in ideal cases [8]. A *tile set* consists of a finite set of unique tiles that are used to self-assemble into a DNA *aggregate*. A tile is assumed to be square and tiles can not rotate by assumption. Each of the four sides of a tile has a *bond type*. The bond types of a tile determine the uniqueness of the tile. Each bond type has an associated *bond strength*, i.e., either 0, 1 or 2. Two bonds of the same type can bond together, with a corresponding bond strength. It is assumed that the strength between different bond types is always 0.

In aTAM, self-assembly always begins with a *seed tile*. A tile can be added to the existing aggregate when its total bond strength to the aggregate is greater than or equal to 2. The *Sierpinski triangle tile set* [as shown in Fig. 1(a)] is a widely studied tile set [9] and it is used to illustrate the aTAM model. The four *integers* in a tile denotes the *bond types* of the four sides. The growth of an infinite Sierpinski triangle is shown in Fig. 1(b); smaller tiles represent the correct addition of tiles. In this paper, it is assumed that the correct direction of growth

Manuscript received April 10, 2008; revised August 18, 2008. Current version published January 16, 2009. This paper was recommended by Associate Editor P. Li.

The authors are with the Department of Electrical and Computer Engineering, Northeastern University, Boston, MA 02115 USA (e-mail: xma@ece.neu.edu; lombardi@ece.neu.edu).

Digital Object Identifier 10.1109/TCSII.2008.2010161

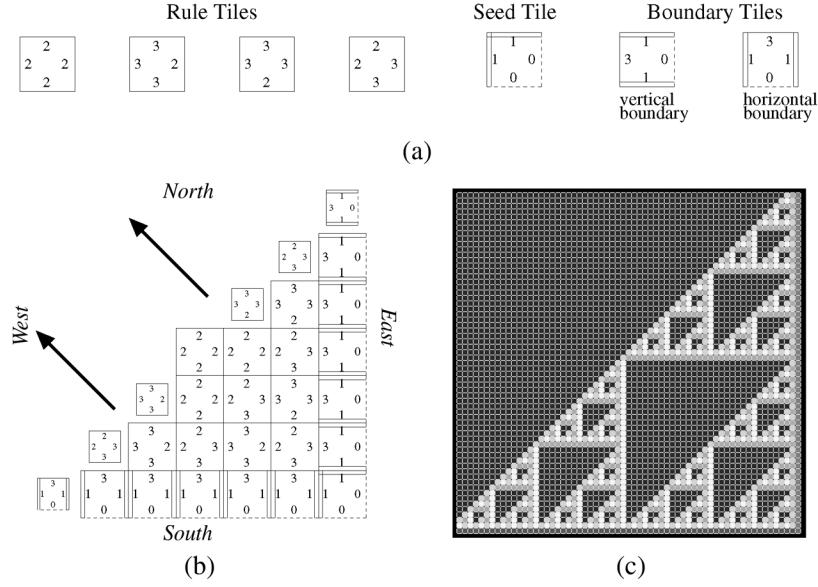


Fig. 1. The Sierpinski Triangle Pattern. (a) Sierpinski tile set. (b) Growth of tile set. (c) Assembled pattern.

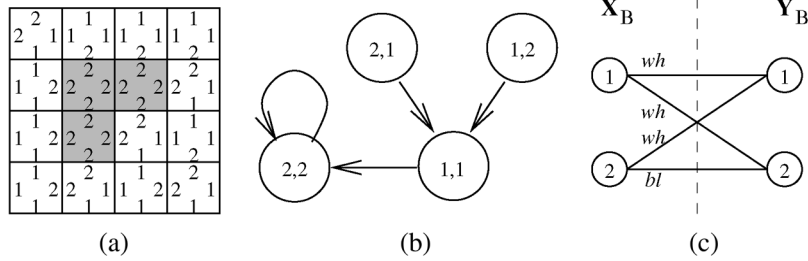


Fig. 2. Graph model for the Sierpinski triangle tile set. (a) Pattern and tile set (Sierpinski triangle). (b) Tile graph. (c) Bond graph.

for a tile set is from south–east to north–west, as indicated by arrows in Fig. 1(b). The assembled pattern is shown in Fig. 1(c). The design of the tile set presented in this paper is based on aTAM as model for DNA self-assembly; this model has been extensively analyzed in the technical literature. It has restricted applicability due to the assumptions of non rotating tiles and distinct bonds and considers the simple yet ideal scenario of self assembly. A tool *Xgrow* [8] has been developed to simulate DNA self-assembly.

III. PROBLEM DEFINITION

For characterizing the optimization problem for tile set synthesis, we first define the process by which a tile set can produce a desired pattern.

Definition 1: If following the self-assembly rules defined in the aTAM model, a tile set T can construct a finite-size pattern P , then T is said to be the *assembling tile set* of P .

In this paper, a *graph model* is used to represent the tile set. An example of a tile set and its model (consisting of two graphs) are shown in Fig. 2. In the *tile graph* $G_T = (V, E)$, each tile type is represented by a directed edge in E . The edge connects two vertices in V , (B_e, B_s) and (B_w, B_n) , where B_e , B_s , B_w and B_n are the bond types on the tile's east, south, west and north, respectively. If the bond types of the four sides of a tile are denoted by east = $e1$, south = $s1$, west = $w1$, north = $n1$, then this tile is represented by a directed edge from vertex $v1 = (e1, s1)$

to vertex $v2 = (w1, n1)$. An example is shown in Fig. 2(a). The tile in the lower right corner has the following bond types: east = 1, south = 1, west = 2, north = 2. As shown in Fig. 2(b), it is represented by a directed edge from vertex (1,1) to vertex (2,2). The *bond graph* $G_B = (X_B, Y_B, E_B)$ is an undirected bipartite graph, in which bond types are represented by vertices. For any vertex (x, y) in the tile graph G_T , there is an edge connecting the vertex representing the horizontal bond x in the set X_B (referred to as the *left side* of the bond graph) and the vertex for the vertical bond y in the set Y_B (referred to as the *right side* of the bond graph). E_B is a set of colored edges. For an edge (x', y') in the bond graph, if there exists a tile in the tile set with east side of bond type x' and south side of bond type y' , then the color of the tile (**wh** or **bl**, for white or black) is assigned to this edge. If such a tile does not exist, then this edge has no color. Not all graph models generated by the above definition can define a tile set that will generate a deterministic pattern.

Definition 2: A tile set is valid iff: 1) the *out-degree* of any vertex in the tile graph is not greater than 1 and 2) no edge in the bond graph is assigned both colors (**wh** and **bl**).

In a “trivial” tile set, each bond in the horizontal (east and west) and vertical (north and south) sides has a unique type and every pixel in the pattern corresponds to a unique tile type. So by definition, it is guaranteed that any finite-size pattern has at least one trivial tile set as its valid assembling tile set. Also, as every

bond/tile in the pattern corresponds to a unique bond/tile type, any two trivial tile sets can be converted to each other by simply substituting their notation. The following Lemma is therefore applicable.

Lemma 1: All different designs of a trivial tile set are different representations of the same design.

Starting from a trivial tile set of P , tile sets with reduced number of tile types can be generated by merging bond types in P . The process of merging bond types takes two nodes on the same side of the bond graph, makes them into one node (i.e., merging) and move all edges ending on either nodes to the new node. If merging of the nodes results in merging of edges in the bond graph, then each new edge is the result of merging two edges. The color assigned to the new edge is established by the colors of the two edges prior to merging. This is given as follows: if both edges have the same color assignment (**wh**, **bl** or no color), then the new edge has the same color assignment; if only one of the edges has **wh** or **bl** color, then the new edge is assigned that color; if the edges have different colors, then the new edge has both colors (which will result in an invalid tile set). The nodes in the tile graph need to be changed accordingly if the edges in the bond graph are merged. It will be shown next that the process of merging bonds can generate a tile set that assembles the same pattern as the tile set before merging.

Definition 3: Assume that T_1 is an assembling tile set for P and T_2 is a valid tile set resulting from a series of bond-type merges in T_1 . A partial assembly As_1 from T_1 and a partial assembly As_2 from T_2 are said to be *equivalent* if 1) They have the same shape, i.e., any empty location in As_1 is also empty in As_2 and vice versa. 2) For any bond in the assemblies, the bond type $B_{2,x}$ found on As_2 is the result of merging bond type $B_{1,y}$ (the bond type found at the same location on As_1) with other bond types.

For any bond type $B_{1,i}$ in T_1 , a bond type $B_{2,j}$ in T_2 can be found such that $B_{2,j}$ is the result of merging $B_{1,i}$ with other bond types in T_1 . $B_{1,i}$ is referred to as the *merging source*. For any tile in T_1 represented by an edge $((B_{e1}, B_{s1}), (B_{w1}, B_{n1}))$ in the tile graph, a tile can be found in T_2 that is represented in the tile graph by the edge $((B_{e2}, B_{s2}), (B_{w2}, B_{n2}))$ such that B_{e1}, B_{s1}, B_{w1} and B_{n1} are the merging sources of B_{e2}, B_{s2}, B_{w2} and B_{n2} , respectively. The tile $((B_{e1}, B_{s1}), (B_{w1}, B_{n1}))$ is referred to as the merging source of tile $((B_{e2}, B_{s2}), (B_{w2}, B_{n2}))$.

Theorem 1: Starting from an assembling tile set T_1 for pattern P , if the outcome of a series of bond type merging operations is a valid tile set, then the new tile set T_2 is also an assembling tile set for P .

Consider Lemma 1; it is evident that any non-trivial tile set can be derived by merging a trivial tile set.

Definition 4: An *optimum tile set* for a pattern of finite size is an assembling tile set for that pattern with the minimum number of tile types.

Definition 5: The *Pattern self-Assembly Tile-set Synthesis (PATS)* consists of finding the optimum tile set for a given finite-size pattern.

Theorem 2: Among all assembling tile sets for P , if the tile graph representing a tile set has the smallest total out-degree among all tile graphs, then this tile set is referred to as having the *minimum tile graph* and is an *optimum tile set* for P .

The objective of PATS is to generate a tile set with a minimum tile graph. The PATS problem can be solved by finding a valid tile graph through a series of bond-merging operations from the trivial tile set. Hereafter, “tile set *synthesis*” refers to the process of merging bond types from a trivial tile set to solve the PATS problem.

IV. COMPLEXITY ANALYSIS

The complexity of the PATS problem is analyzed by relating it to the *minimum graph coloring problem*. The minimum graph coloring problem is known to be NP-complete in the general case (i.e., no restriction on the number of colors). The following process converts the minimum graph coloring problem into the PATS problem in polynomial time. Given a coloring problem with graph $G_c = (V_c, E_c)$, the corresponding PATS problem with a bond graph $G_B = (B_l, B_r, E_B)$ ($B_r \cup B_l$ is denoted by B) and a tile graph $G_T = (T, E_T)$, can be generated as follows.

For each vertex $v_i \in V_c$, generate a pair of vertices $b_{l,i}$ and $b_{r,i}$ in G_B , with $b_{l,i} \in B_l$ and $b_{r,i} \in B_r$. $b_{l,i}$ and $b_{r,i}$ are connected by an edge $e_b^{i,i} = (b_{l,i}, b_{r,i}) \in E_B$. $e_b^{i,i}$ is assigned a color **wh** (note that this color assignment is defined in Section III, it is not the color used in the graph coloring problem). A vertex in the tile graph $t_{i,i} \in T$ is also generated corresponding to the edge $e_b^{i,i}$. $t_{i,i}$ has a self-loop edge $e_t^{i,i} = (t_{i,i}, t_{i,i}) \in E_T$. For each edge $e_j = (v_{j1}, v_{j2}) \in E_c$, generate an edge $e_b^{j1,j2} = (b_{l,j1}, b_{r,j2})$ and an edge $e_b^{j2,j1} = (b_{l,j2}, b_{r,j1})$ in the bond graph and the corresponding vertices $t_{j1,j2} \in T$ and $t_{j2,j1} \in T$ in the tile graph. The colors of $e_b^{j1,j2}$ and $e_b^{j2,j1}$ are **bl**. The edges $e_t^{j1,j2} = (t_{j1,j2}, t_{j2,j1})$ and $e_t^{j2,j1} = (t_{j2,j1}, t_{j1,j2})$ connect the two vertices $t_{j1,j2}$ and $t_{j2,j1}$ into a loop.

An example of generating PATS from a coloring problem is shown in Fig. 3. The bond and tile graphs generated by the above process have the following properties.

- 1) For any vertex $b_{l,i} \in B_l$, there must be a vertex $b_{r,i} \in B_r$ and vice versa. There must be an edge $e_b^{i,i} \in E_B$ and its color must be **wh**. The vertex $t_{i,i}$ in the tile graph is connected and only connected by the edge $(t_{i,i}, t_{i,i})$.
- 2) For any edge $e_b^{i,j}$ (where $i \neq j$), there must be an edge $e_b^{j,i}$ in the graph. Both edges are assigned **bl** as color. The vertex $t_{i,j}$ in the tile graph is connected and only connected by edge $(t_{i,j}, t_{j,i})$ and edge $(t_{j,i}, t_{i,j})$.

These properties defines a feature that hereafter it is referred to as the *feature-S*. The following lemmas prove that an optimum solution to the coloring problem can be generated from an optimum solution to its corresponding PATS problem.

Lemma 2: For the PATS problem generated from the coloring problem, any optimum solution can be converted into an optimum solution with a complete bipartite graph as its bond graph.

The optimum solution to the PATS problem can be attained by a series of node merging operations in the bond graph. If a node merging in the bond graph is not related to an edge merging, then it can be omitted and the solution will have the same number of tile types. After omitting all bond merges of this type from an optimum solution, the resulting solution is also optimum. The new solution can be attained from the original graph of the PATS problem through a series of edge merges in the bond

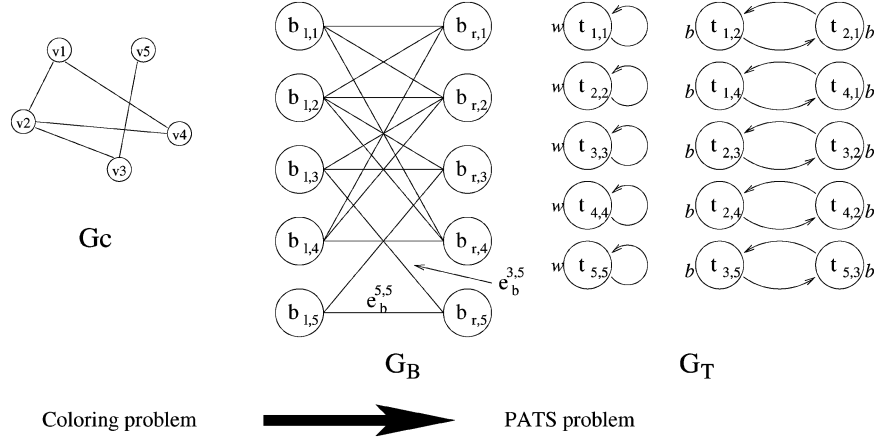


Fig. 3. Example of converting the coloring problem to the PATS problem.

graph. This solution is referred to as the *edge merged optimum solution* for the PATS problem. According to the color assignment rule for merging, if the bond graph of the original PATS problem generated from the coloring problem has the *feature-S*, then the bond graph after each edge merging operation still satisfies the *feature-S*. Therefore, after a series of edge merges in the bond graph, the *edge merged optimum solution* must have a bond graph with the *feature-S*. Due to the *feature-S*, if the bond graph of an *edge merged optimum solution* T_{opt} is a complete graph, then it must also be a complete bond graph.

Lemma 3: For the coloring problem and the PATS problem derived from it, any solution to the PATS problem can be converted into a solution to the coloring problem (and vice versa).

Let the coloring problem be denoted by $\mathbf{P}_c$ and the PATS problem by $\mathbf{P}_p$. $\mathbf{P}_p$ is represented by a graph $G_B = (B_l, B_r, E_B)$ and $G_T = (T, E_T)$. Given a solution to $\mathbf{P}_p$, a solution to $\mathbf{P}_c$ can be generated. First, the solution of $\mathbf{P}_p$ is changed into a solution with the *feature-S*. Using the previously presented method for generating an *edge merged optimum solution*, it is guaranteed that the conversion is possible and the solution with the *feature-S* (represented by the graph $G'_B = (B'_l, B'_r, E'_B)$ and $G'_T = (T', E'_T)$) has the same number of tile types as prior to conversion. A solution for $\mathbf{P}_c$ can always be generated as follows: (1) For any vertex $b'_{l,i} \in B'_l$, find its merging sources in B_l . (2) Select all vertices in G_c (the graph for the coloring problem) that correspond to the merging sources of $b'_{l,i}$. All these vertices are assigned the same color in the solution for $\mathbf{P}_c$. (3) Assign a different color each time a different vertex in B'_l is used, until all vertices in B'_l have been processed. The solution generated by the above process guarantees that every vertex in G_c is assigned a color and no two vertices are assigned the same color if they are connected by an edge. So, the solution is a valid solution for $\mathbf{P}_c$, and $|B'_l|$ colors are used in this solution.

Given a solution to $\mathbf{P}_c$, a solution to $\mathbf{P}_p$ can be generated. The solution to $\mathbf{P}_c$ is given by the subsets $(S_1, S_2, \dots, S_k)$, in which each subset includes all vertices with the same color. For all vertices $(v_{i1}, v_{i2}, \dots, v_{ij})$ in subset $S_i \in (S_1, S_2, \dots, S_k)$, their corresponding nodes in G_l (denoted by $b_{l,i1}, b_{l,i2}, \dots, b_{l,ij}$) are merged and their corresponding nodes in G_r ($b_{r,i1}, b_{r,i2}, \dots, b_{r,ij}$) are merged.

After merging all subsets in $(S_1, S_2, \dots, S_k)$, the resulting graphs G'_B and G'_T represent a solution to $\mathbf{P}_c$.

Lemma 4: The solution to $\mathbf{P}_c$ that is generated from the optimum solution to $\mathbf{P}_p$, is optimum.

Let the optimum PATS solution be represented by $G_B = (B_l, B_r, E_B)$, $G_T = (T, E_T)$. According to the method for generating the coloring solution from the PATS solution, the solution G_B, G_T is first converted into the solution G'_B, G'_T , which is also optimum and satisfies the *feature-S*. Then, the coloring solution is generated from G'_B, G'_T . The coloring solution is denoted by $\mathbf{S}$. If $\mathbf{S}$ is not the optimum solution to $\mathbf{P}_c$, then there must be an optimum solution (denoted by $\mathbf{S}'$) with a smaller number of colors. From $\mathbf{S}'$, the other PATS solution (represented by $G''_B = (B''_l, B''_r, E''_B)$, $G''_T = (T'', E''_T)$) can be generated. It follows that $B''_l < B'_l$ and $B''_r < B'_r$. As G'_B is a complete graph in the optimum PATS solution, then $|E'_B|$ must be smaller than $|E''_B|$. Every edge in E'_B corresponds to a tile type and the number of tile types in G''_B, G''_T is not greater than $|E''_B|$. Therefore, the number of tile types in the solution G''_B, G''_T must be smaller than the solution G'_B, G'_T . This contradicts the assumption that G'_B, G'_T is an optimum solution for $\mathbf{P}_p$. Based on Lemmas 2–4, it can be proved that the minimum coloring problem can be reduced to the PATS problem.

Theorem 3: The PATS problem is equivalent to the coloring problem and therefore in general, it is NP-complete.

V. CONCLUSION

In this paper, the synthesis problem [4] for generating a tile set for a finite-size pattern (referred to as the Pattern self-Assembly Tile-set Synthesis, PATS) has been analyzed as a combinatorial optimization problem. It has been shown that the PATS problem is equivalent to the minimum graph coloring problem, hence in general, it is NP-complete. Future research is currently being pursued to investigate efficient heuristic algorithms for tile synthesis; these algorithms utilize the graph model presented in this manuscript together with additional transformations that relate the tiles (rule, boundary and seed) to isomorphic graphs. It is envisioned that through graph isomorphism further reduction in tile set cardinality can be accomplished.

REFERENCES

- [1] R. Compano, L. Molenkamp, and D. J. Paul, "Technology roadmap for nanoelectronics," in Eur. Commission IST programme, Future and Emerging Technologies, 2000 [Online]. Available: cordis.europa.eu/ist/fet/nidqf.htm
- [2] C. Lin, Y. Liu, S. Rinker, and H. Yan, "DNA tile based self-assembly: Building complex nanoarchitectures," *Chem. Phys. Chem.*, vol. 7, pp. 1641–1647, 2006.
- [3] S.-H. Park, C. Pistol, S. J. Ahn, J. H. Reif, A. R. Lebeck, C. Dwyer, and T. H. LaBean, "Finite-size, fully-addressable DNA tile lattices formed by hierarchical assembly procedures," *Angewandte Chemie, Intl. Ed. in English*, vol. 45, no. 5, pp. 735–739, Jan. 2006.
- [4] X. Ma and F. Lombardi, "Synthesis of tile sets for DNA self-assembly," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 5, pp. 963–967, May 2008.
- [5] W. Hu, K. Sarveswaran, L. M. , and G. H. Bernstein, "High-resolution electron beam lithography and DNA nano-patterning for molecular QCA," *IEEE Trans. Nanotechnol.*, vol. 5, no. 3, pp. 312–316, Sep. 2005.
- [6] E. Winfree, F. Liu, L. A. Wenzler, and N. C. Seeman, "Design and self-assembly of two-dimensional DNA crystals," *Nature*, vol. 394, no. 6693, pp. 539–544, 1998.
- [7] S.-H. Park, H. Yan, J. H. Reif, T. H. LaBean, and G. Finkelstein, "Electronic nanostructures templated on self-assembled DNA scaffolds," *Nanotechnol.*, vol. 15, pp. S525–S527, 2004.
- [8] E. Winfree, Xgrow Homepage [Online]. Available: www.dna.caltech.edu/Xgrow/
- [9] P. W. K. Rothmund, N. Papadakis, and E. Winfree, "Algorithmic self-assembly of DNA Sierpinski triangles," *PLoS Biol.*, vol. 2, no. 12, p. e424, 2004.